DATA ANALYTICS

DATA MANAGEMENT, DATA VIZ & TEXT MINING

Diplôme Sorbonne Data Analytics

DATA VISUALISATION

Avec Matplotlib

Matplotlib

- La visualisation des données est essentielle pour comprendre rapidement les tendances et les relations dans votre jeu de données.
- Matplotlib est l’une des bibliothèques les plus populaires pour tracer avec Python.
- Matplotlib est connu comme le « grand-père » des bibliothèques de traçage et de visualisation pour Python.
- De nombreuses autres bibliothèques de visualisation sont construites directement à partir de Matplotlib .

Matplotlib

- Matplotlib est fortement inspiré par les fonctions de traçage du langage de programmation MatLab.
- Elle permet la création de presque n’importe quel type de plot et une forte personnalisation.
- Cette possibilité de personnaliser fortement un graph est un compromis pour les débutants, car il peut être déroutant d’apprendre la syntaxe Matplolib au début.
- Cela est principalement dû au fait qu’il existe en fait deux approches distinctes pour créer des graphes, des méthodes fonctionnelles et des méthodes POO.

Matplotlib

- Cette section de Matplotlib cherche à dissiper toute confusion en séparant clairement ces deux approches.

➢ ***Matplotlib Basics*** : présentation des Figures

➢ ***Méthodes de création de graphiques :*** approche fonctionnelle et approche orientée objet (POO, notamment pour les sous-lots)

Matplotlib

Dans cette section, nous ferons référence à la documentation en ligne de Matplotlib :

- https://matplotlib.org/

Ainsi que la galerie d'exemples de graphes et de codes (très utile!)

- https://matplotlib.org/gallery.html

Matplotlib

Deux objectifs principaux avec Matplotlib :

- Être capable de tracer une relation fonctionnelle :
  - y = 2x
- Être capable de tracer une relation entre les points de données brutes :
  - x = [1,2,3,4]
  - y = [2,4,6,8]

LES BASIQUES

Les basiques

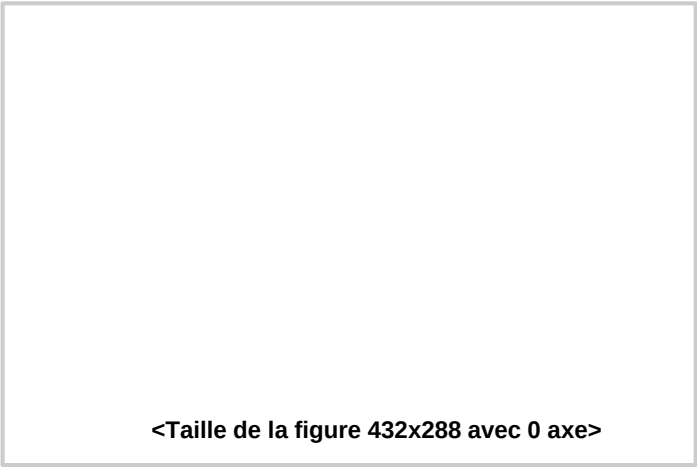
- La façon la plus simple d'utiliser Matplotlib est d'appeler les tracés de fonctions :
  - **plt.plot(x,y)**
- Ces appels de fonction sont simples à utiliser, mais ne permettent pas des niveaux de contrôle très élevés.

- Construisons rapidement une compréhension visuelle de l'objet Figure avant de le coder avec Python…

- <u>Remarque</u> :
  - L'objet Figure que nous allons montrer n'est techniquement pas visible tant que vous n'y ajoutez pas d'axes.

Les basiques

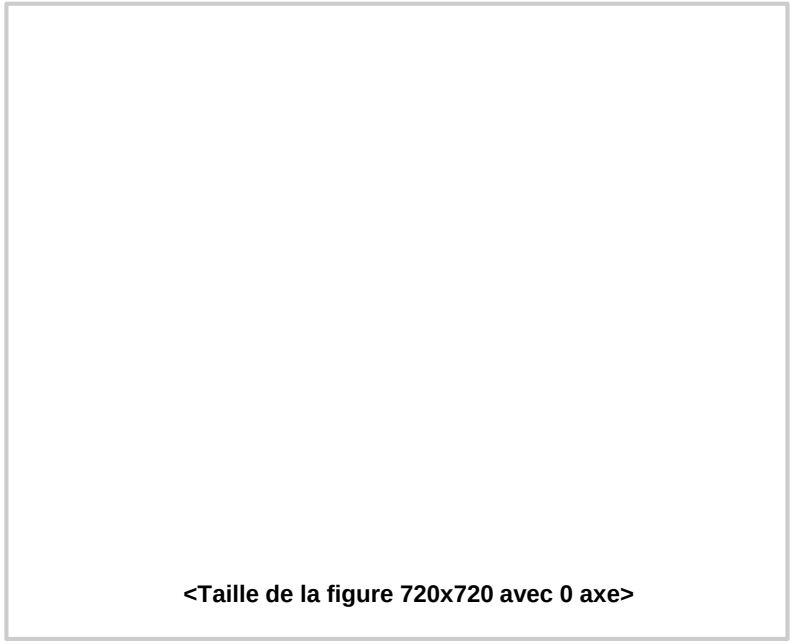
- plt.figure()



Création d’une figure vide

Les basiques

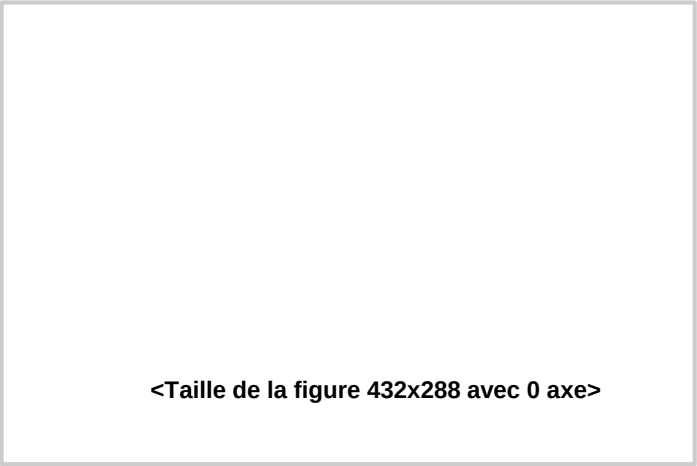
- plt.figure(figsize=(10,10)) # taille en pouce



Personnalisation de la taille

Les basiques

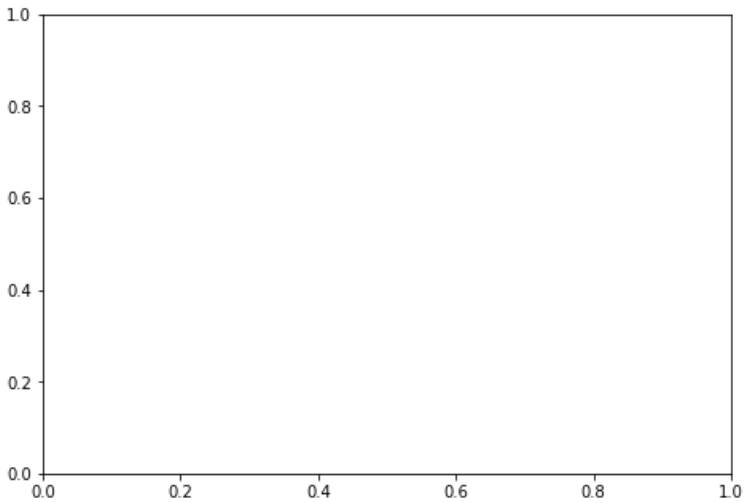
- fig = plt.figure()
- Toile vierge, en attente d’un ensemble d’axes pour tracer.



Les basiques

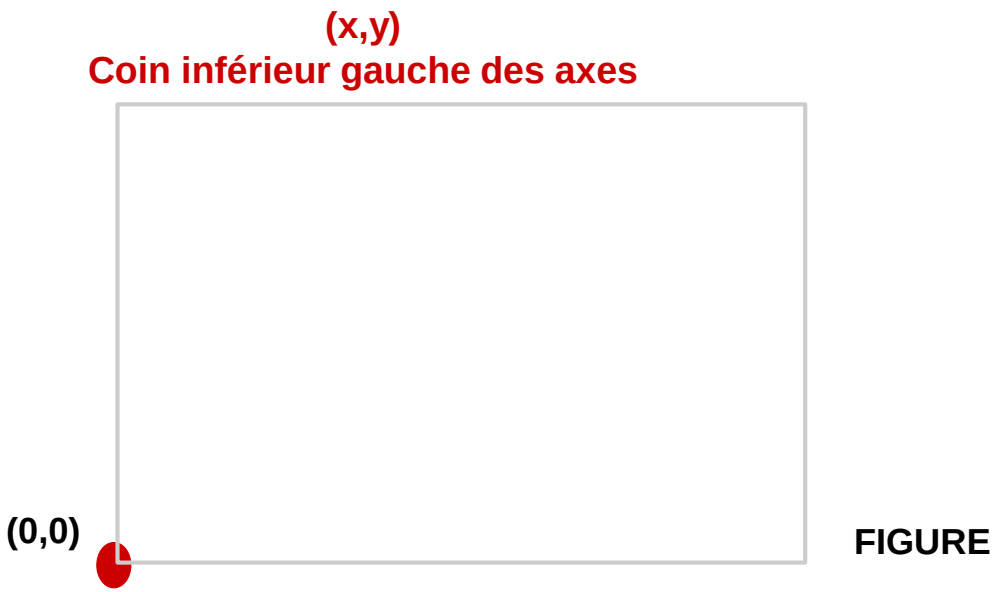
- fig = plt.figure()
  axes = fig.add_axes([0, 0, 1, 1])

=> ajouter manuellement un axe (ou système de coordonnées) à la figure fig



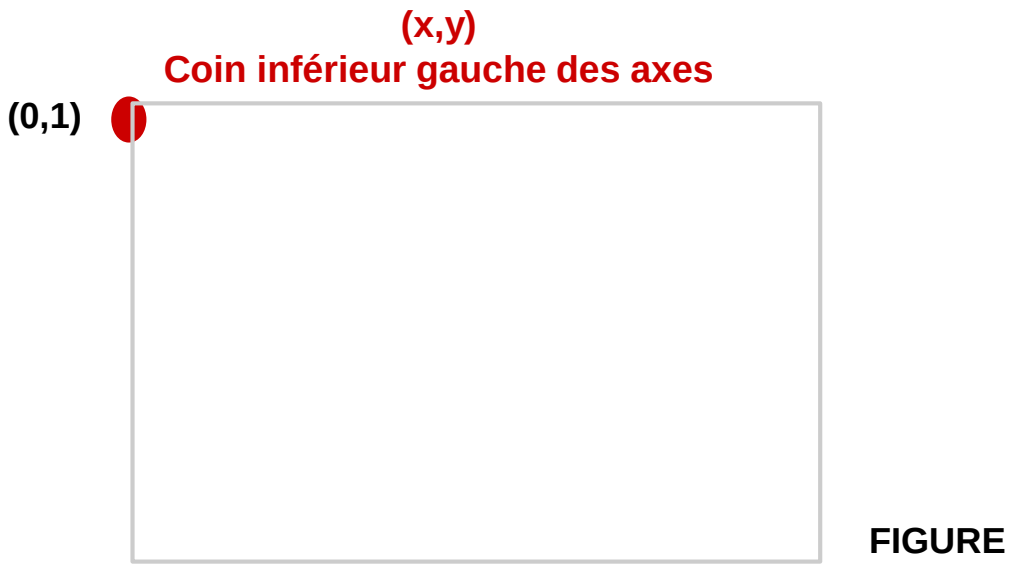
Les basiques

- fig = plt.figure()
  axes = fig.add_axes([0, 0, 1, 1])



Les basiques

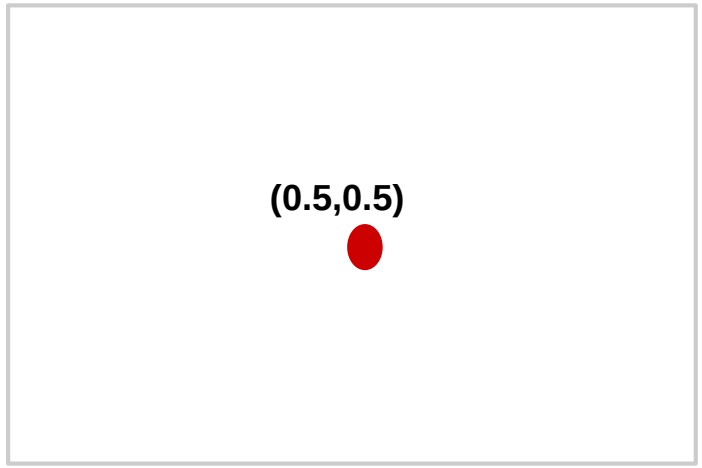
- fig = plt.figure()
  axes = fig.add_axes([0, 1, 1, 1])



Les basiques

- fig = plt.figure()
  axes = fig.add_axes([0.5, 0.5, 1, 1])

(x,y)
Coin inférieur gauche des axes

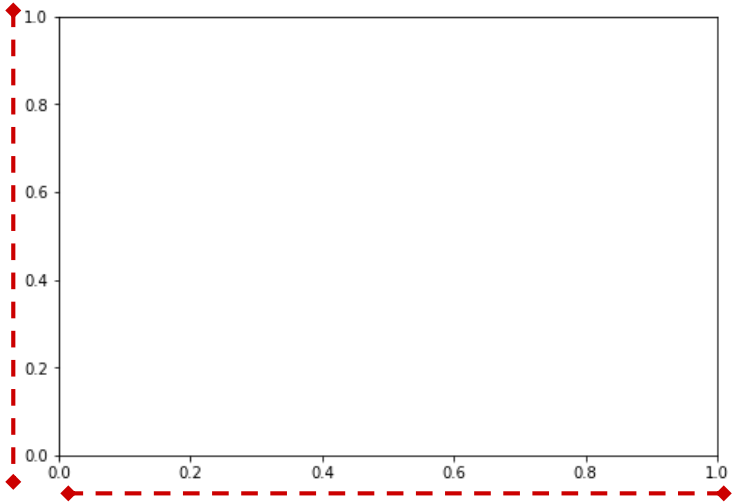


FIGURE

Les basiques

- fig = plt.figure()
  axes = fig.add_axes([0, 0, 1, 1])

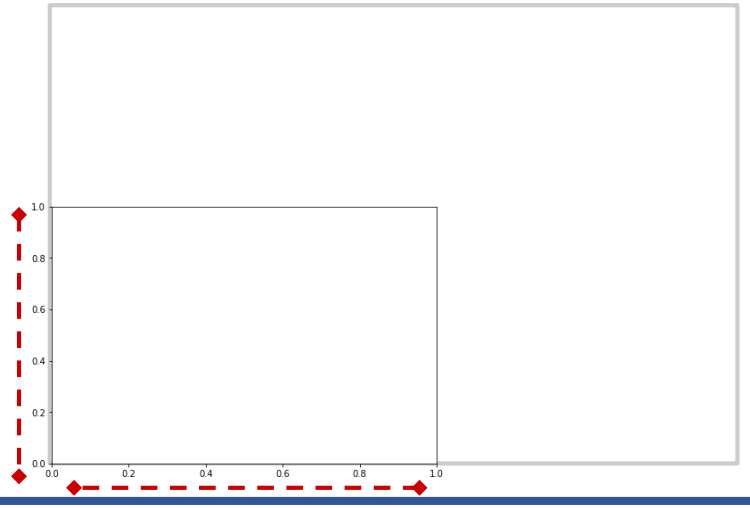
(largeur, hauteur) des axes



Les basiques

- fig = plt.figure()
  axes = fig.add_axes([0, 0, 0.5, 0.5])

(largeur, hauteur)
des axes

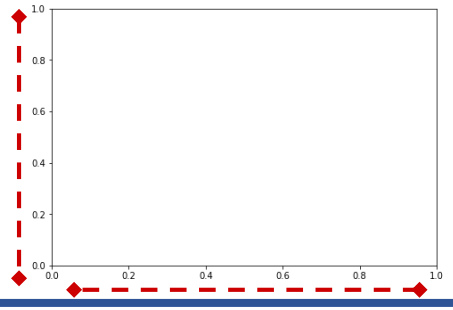


FIGURE

Les basiques

- fig = plt.figure()
  axes = fig.add_axes([0, 0, 0.5, 0.5])

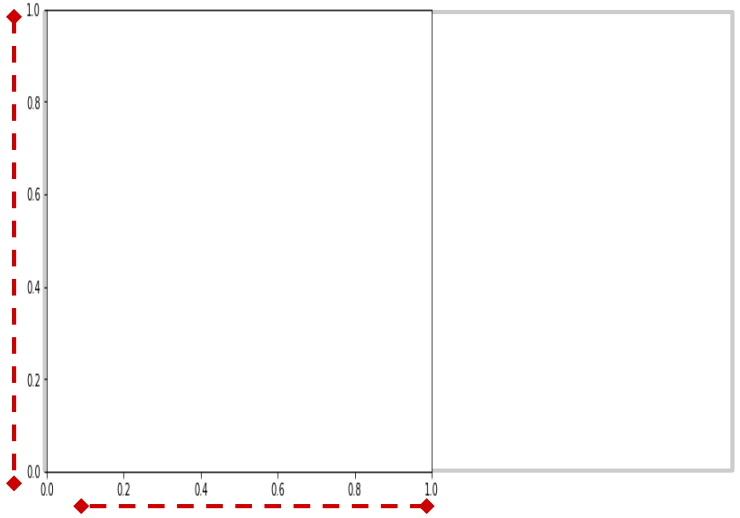
(largeur, hauteur)
des axes



Les basiques

- fig = plt.figure()
  axes = fig.add_axes([0, 0, 0.5, 1])

(largeur, hauteur)
des axes

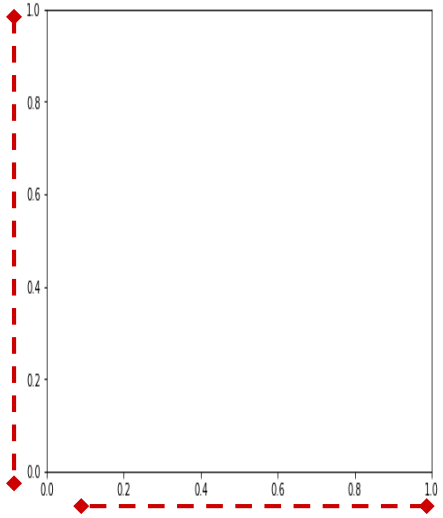


FIGURE

Les basiques

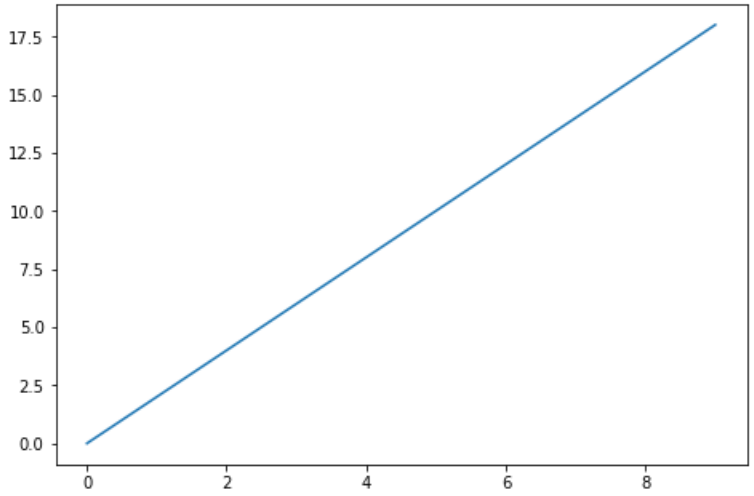
- fig = plt.figure()
  axes = fig.add_axes([0, 0, 0.5, 1])

(largeur, hauteur) des axes



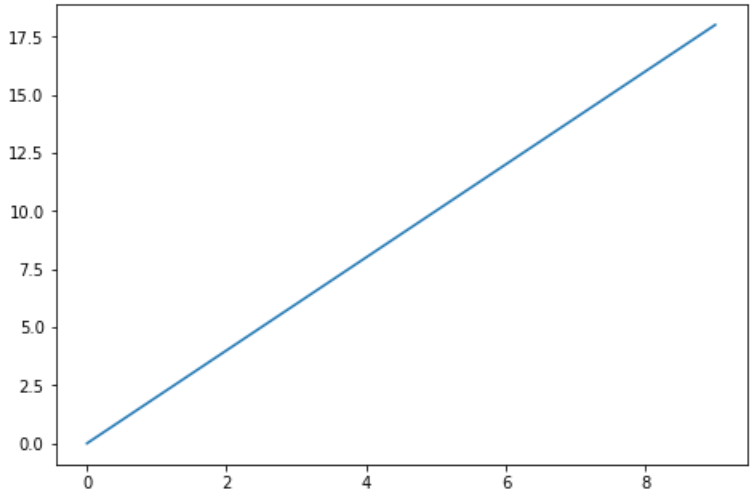
Les basiques

- fig = plt.figure()
  axes = fig.add_axes([0, 0, 1, 1])
  axes.plot(x, y)



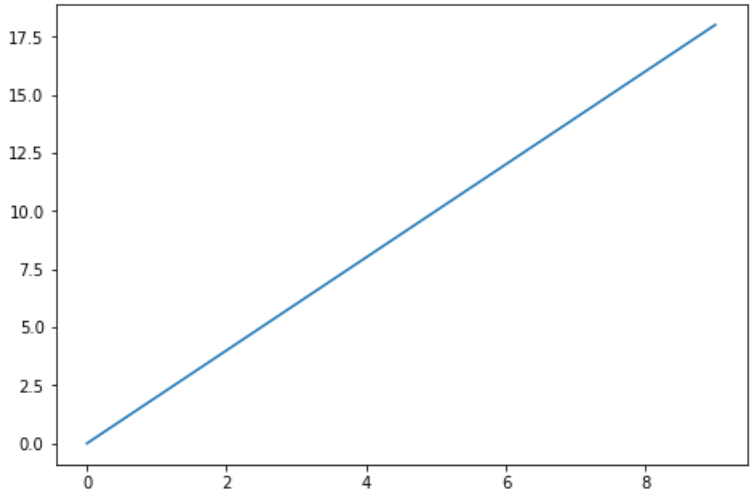
Les basiques

- fig = plt.figure()
  axes = fig.add_axes([0, 0, 1, 1])
  axes.plot(x, y)



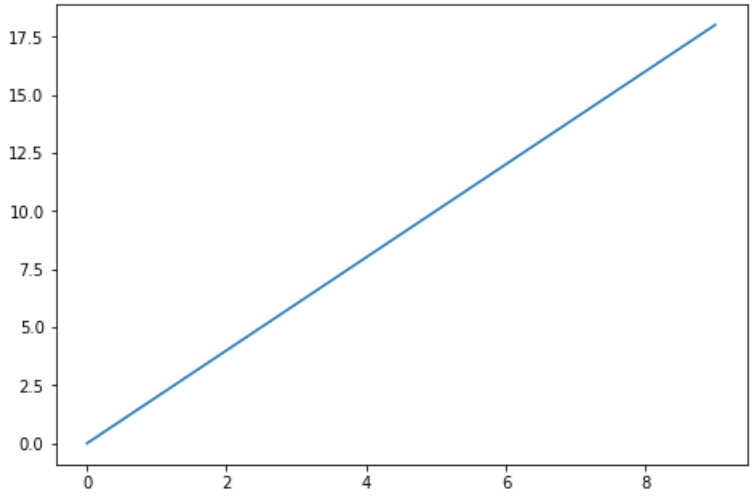
Les basiques

- fig = plt.figure()
  axes = fig.add_axes([0, 0, 1, 1])
  axes.plot(x, y)



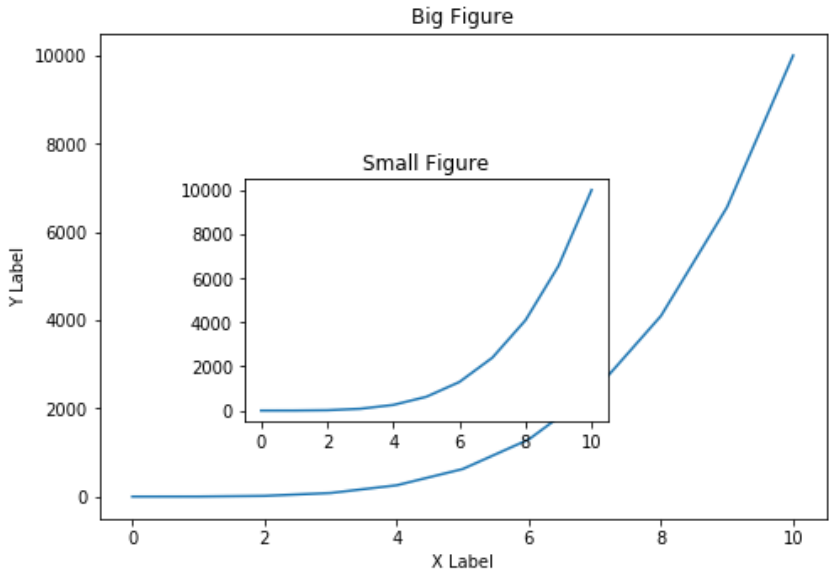
Les basiques

- fig = plt.figure()
  axes = fig.add_axes([0, 0, 1, 1])
  axes.plot(x, y)



Les basiques

- Cette méthodologie nous permet d'ajouter plusieurs axes ainsi que de déplacer et redimensionner les axes.



Notebook 1 : Les basiques

Notebook 2 : Les figures

Matplotlib SubPlots

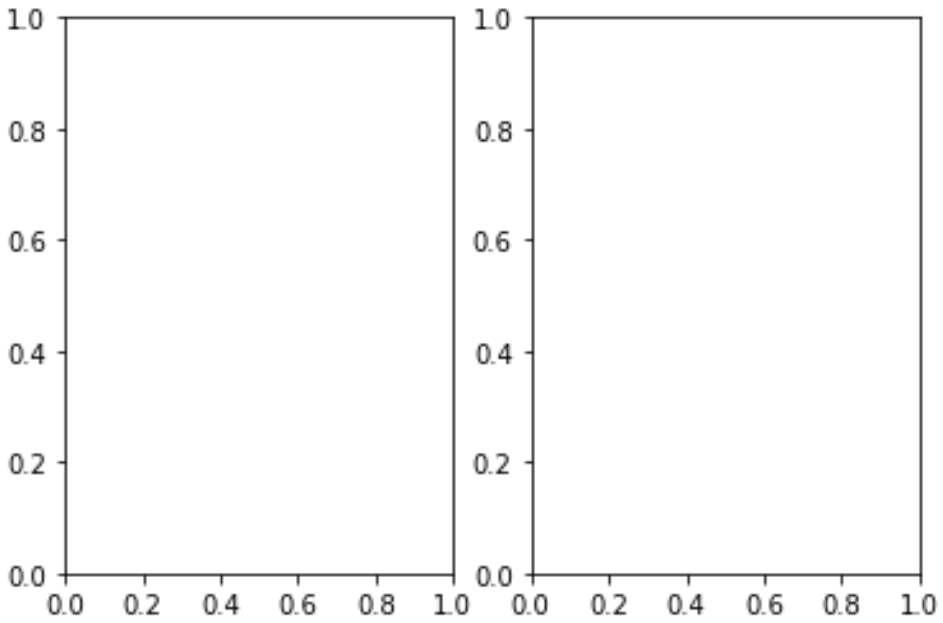
Matplotlib SubPlots

- En théorie, nous pourrions créer un objet Figure, puis ajouter et organiser manuellement des ensembles d'axes pour aligner plusieurs tracés côte à côte.
- Cependant, Matplotlib est livré avec une fonction pré-**configurée appelée plt.subplots()** qui le fait automatiquement pour nous!
- L'appel plt.subplots() nous permet de créer facilement des objets Figure et Axes dans des formats côte à côte.
- La commande plt.subplots() retourne un tuple contenant le canevas Figure, puis un tableau numpy contenant les objets axes.

Matplotlib SubPlots

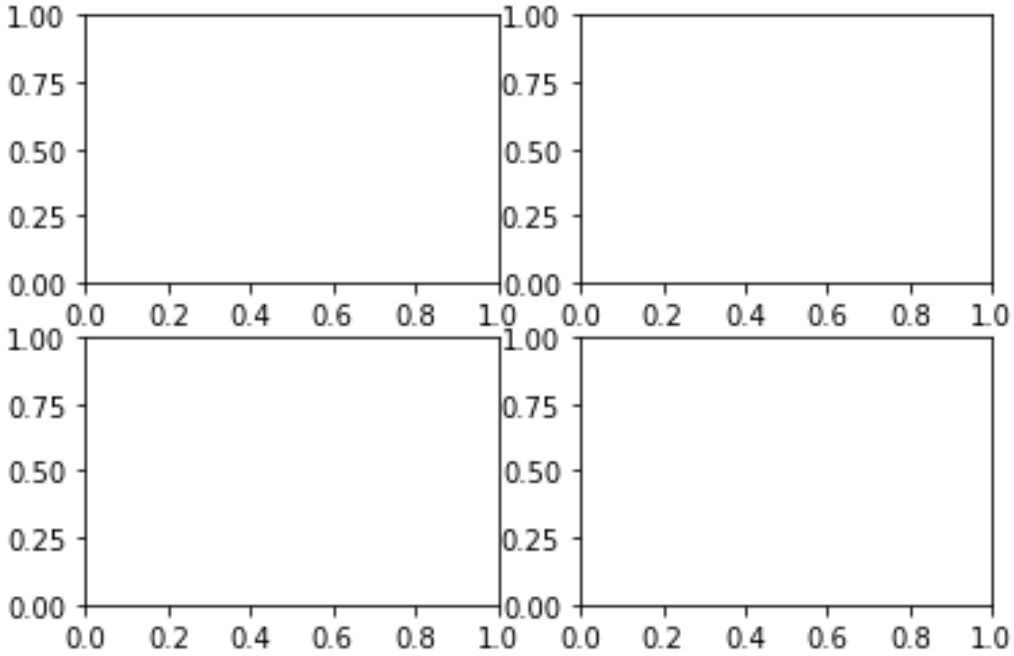
- fig, axes = plt.subplots(nrows=1, ncols=2)

nrows=1 → 1 ligne de graphiques
ncols=2 → 2 colonnes de graphiques



Matplotlib SubPlots

- fig, axes = plt.subplots(nrows=2, ncols=2)



Matplotlib SubPlots

- plt.subplots() renvoie un tuple qui, par convention commune, est étiqueté **(fig,axes) :**

  - **figure**

    C’est toute la toile de Figure.

  - **axes**

    Il s’agit d’un tableau numpy contenant chacun des axes en fonction de la position dans le canevas global.

Notebook 3 : Les subplots

Styling

Styling

- Matplotlib offre des fonctions de style très robustes qui nous permettent d'éditer des couleurs, des légendes, des largeurs de lignes, des marqueurs et bien plus encore!
- Principaux styles abordés :
  - Légendes
  - Style visuel
    - Couleurs
    - Modification des lignes
      - ✓ Couleurs, largeurs, styles
    - Modification des marqueurs
      - ✓ Couleurs, taille, styles, bords

Notebook 4 : Styling

Notebook 5 : Exercices